

# 0) Final project layout

```
spark-kafka-demo/
├── docker-compose.yml
├── spark/
│   ├── app/
│   │   ├── stream_job.py
│   │   └── requirements.txt          # (optional; can be empty)
│   └── output/                     # created automatically by Spark
│       (parquet + checkpoints)
├── producer/
│   ├── Dockerfile
│   └── produce.py
```

Create the folders now; we'll fill the files below.

---

## 1) `docker-compose.yml`

This composes Kafka, Kafka UI, Spark (master/worker), the streaming app (spark-submit), and a producer.

```
version: "3.8"
```

```
networks:
```

```
    demo-net:
```

```
volumes:
```

```
    kafka_data:
```

```
    spark_output:
```

```
services:
```

```
    # ----- Kafka (KRaft combined mode, no ZooKeeper) -----
```

```
    kafka:
```

```
        image: confluentinc/cp-kafka:7.7.0
```

```
        container_name: kafka
```

```
        networks: [demo-net]
```

```
ports:

  - "9092:9092"

environment:

  # KRaft single-node (combined) settings

  KAFKA_PROCESS_ROLES: "broker,controller"

  KAFKA_NODE_ID: "1"

  KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"

  KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:9092"

  KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"

  KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"

  KAFKA_LOG_DIRS: "/var/lib/kafka/data"

  KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"

  # Required in KRaft mode

  CLUSTER_ID: "4L6g3nShT-eMCtK--X86sw"

volumes:

  - kafka_data:/var/lib/kafka/data

# ----- Kafka UI -----

kafka-ui:

  image: provectuslabs/kafka-ui:latest

  container_name: kafka-ui

  networks: [demo-net]

  ports:

    - "8080:8080"

  environment:

    KAFKA_CLUSTERS_0_NAME: "local"

    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: "kafka:9092"
```

```

    depends_on:

      - kafka

# ----- Spark Master (Docker Official Image) -----

spark-master:

  image: spark:3.5.1-python3

  container_name: spark-master

  networks: [demo-net]

  hostname: spark-master

  ports:

    - "7077:7077"    # Spark Master RPC

    - "8081:8080"    # Web UI (mapped to localhost:8081)

  command: >

    bash -lc "/opt/spark/bin/spark-class
org.apache.spark.deploy.master.Master

    --host spark-master --port 7077 --webui-port 8080"

  depends_on:

    - kafka

# ----- Spark Worker -----

spark-worker:

  image: spark:3.5.1-python3

  container_name: spark-worker

  networks: [demo-net]

  hostname: spark-worker

  ports:

    - "8082:8081"    # Worker UI (mapped to localhost:8082)

  environment:

```

```

    - SPARK_WORKER_CORES=2

    - SPARK_WORKER_MEMORY=2g

command: >

    bash -lc "/opt/spark/bin/spark-class
org.apache.spark.deploy.worker.Worker

    spark://spark-master:7077 --webui-port 8081"

depends_on:

    - spark-master

# ----- Spark App (runs spark-submit once) -----

spark-app:

    image: spark:3.5.1-python3

    container_name: spark-app

    networks: [demo-net]

    depends_on:

        - spark-master

        - spark-worker

        - kafka

    volumes:

        - ./spark/app:/opt/spark-app

        - spark_output:/opt/spark/output

# Wait a bit, then submit the Structured Streaming job

command: >

    bash -lc "

        sleep 10 &&

        /opt/spark/bin/spark-submit

        --master spark://spark-master:7077

        --packages org.apache.spark:spark-sql-kafka-0-
10_2.12:3.5.1,org.apache.kafka:kafka-clients:3.7.0

```

```

/opt/spark-app/stream_job.py

"

# ----- Simple JSON Producer -----

producer:

  build: ./producer

  container_name: producer

  networks: [demo-net]

  depends_on:

    - kafka

  environment:

    - KAFKA_BOOTSTRAP=kafka:9092

    - KAFKA_TOPIC=events

```

### Why these versions?

- Spark 3.5.1 + spark-sql-kafka-0-10\_2.12:3.5.1 match.
- Kafka clients 3.7.0 are compatible with Kafka 3.7.0 broker.
- KRaft mode avoids ZooKeeper (simpler for students).

---

## 2) Producer files

### producer/Dockerfile

```

FROM python:3.11-slim

WORKDIR /app
RUN pip install --no-cache-dir kafka-python==2.0.2

COPY produce.py /app/produce.py

CMD ["python", "/app/produce.py"]

```

### producer/produce.py

```

import json, os, random, time
from datetime import datetime, timezone
from kafka import KafkaProducer

bootstrap = os.getenv("KAFKA_BOOTSTRAP", "kafka:9092")

```

```

topic = os.getenv("KAFKA_TOPIC", "events")

producer = KafkaProducer(
    bootstrap_servers=bootstrap,
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    linger_ms=50,
)

regions = ["EU", "US", "APAC"]
types = ["click", "view", "purchase"]

print(f"Sending events to {bootstrap} topic '{topic}' ...")
i = 0
while True:
    i += 1
    event = {
        "id": i,
        "ts": datetime.now(timezone.utc).isoformat(),
        "type": random.choice(types),
        "amount": round(random.uniform(1, 500), 2),
        "region": random.choice(regions),
    }
    producer.send(topic, value=event)
    if i % 20 == 0:
        producer.flush()
        print(f"Sent {i} messages")
        time.sleep(0.2) # ~5 msgs/sec

```

**What it does:** emits simple JSON events forever to topic events.

---

### 3) Spark Streaming job

**spark/app/stream\_job.py**

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StructField, IntegerType,
StringType, DoubleType, TimestampType

# 1) Create Spark session
spark = (
    SparkSession.builder
        .appName("KafkaToSparkStreaming")
        .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

# 2) Define schema for the JSON inside Kafka "value"
event_schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("ts", StringType(), True),          # parse later or keep as
string
    StructField("type", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("region", StringType(), True),

```

```

])

# 3) Read stream from Kafka
df_raw = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "kafka:9092")
        .option("subscribe", "events")
        .option("startingOffsets", "earliest")
        .load()
)

# 4) Parse the Kafka "value" (bytes) as JSON to columns
df_parsed = (
    df_raw
        .selectExpr("CAST(value AS STRING) AS json_str", "timestamp AS kafka_timestamp")
        .select(
            from_json(col("json_str"), event_schema).alias("e"),
            col("kafka_timestamp")
        )
        .select(
            col("e.id").alias("id"),
            col("e.ts").alias("ts"), # ISO string
            col("e.type").alias("type"),
            col("e.amount").alias("amount"),
            col("e.region").alias("region"),
            col("kafka_timestamp")
        )
)

# 5) Write to console (debug) and to parquet (data lake-ish)
# Console sink (nice for learning)
query_console = (
    df_parsed.writeStream
        .format("console")
        .outputMode("append")
        .option("truncate", "false")
        .trigger(processingTime="5 seconds")
        .start()
)

# Parquet sink with checkpointing
query_parquet = (
    df_parsed.writeStream
        .format("parquet")
        .option("path", "/opt/spark/output/parquet/events")
        .option("checkpointLocation",
            "/opt/spark/output/checkpoints/events_to_parquet")
        .outputMode("append")
        .trigger(processingTime="5 seconds")
        .start()
)

# 6) Wait for termination (until container is stopped)
query_console.awaitTermination()
query_parquet.awaitTermination()

```

Notes:

- We parse the **Kafka value** (bytes) into columns using a **JSON schema**.
- We write to **console** (for learning) and to **Parquet** (common in data lakes).
- The **checkpoint** directory is mandatory for exactly-once semantics & recovery.

(Optional) `spark/app/requirements.txt`

You don't strictly need extra Python deps here, but you can leave this file empty or add libs you might use later.

---

## 4) Start everything

From the project root:

```
# 1) Build the producer image
docker compose build producer
```

```
# 2) Start all services
docker compose up -d
```

What to expect:

- **Kafka UI:** `http://localhost:8080`
- **Spark Master UI:** `http://localhost:8081`
- **Spark Worker UI:** `http://localhost:8082`

The `spark-app` service waits ~10s and then runs `spark-submit`.  
The `producer` starts sending JSON events to topic `events`.

---

## 5) Verify it's working

### 1. Kafka UI

- Open `http://localhost:8080`
- Cluster: `local` → Topics → `events` should exist and show incoming messages.

### 2. Spark UIs

- Open `http://localhost:8081` (Master UI)
- See the running application `KafkaToSparkStreaming`.

### 3. Console sink output

- Tail Spark app logs:
- `docker logs -f spark-app`



You should see rows printed every ~5s.

#### 4. Parquet files

- Inspect files written by Spark:
- `docker compose exec spark-app bash -lc "ls -R /opt/spark/output/parquet/events | head -50"`
- Copy data to host (optional):
- `docker cp spark-app:/opt/spark/output ./spark-output`

Then open with any parquet viewer or a Python notebook.

---

## 6) How the pieces fit (short theory)

- **Kafka** stores the stream of records in topic **events**.
  - **Producer** writes JSON messages continuously.
  - **Spark Structured Streaming**:
    - Uses the **Kafka source** (`format("kafka")`) to read micro-batches.
    - Parses the JSON payload with a **schema** for typed columns.
    - Writes results to the **console** and to **Parquet files**.
    - Uses a **checkpoint** to track offsets and guarantee fault tolerance.
- 

## 7) Common student pitfalls & fixes

- **Spark can't talk to Kafka**
    - Ensure `kafka:9092` is reachable (same Docker network).
    - Check `docker compose logs kafka` for errors.
  - **No messages consumed**
    - Verify the producer is running and the topic name matches (`events`).
    - In `stream_job.py`, `startingOffsets="earliest"` helps during first runs.
  - **Package not found (spark-sql-kafka)**
    - We ship it via `--packages` in `spark-submit`. Don't remove it.
    - Internet is required (Maven download) the first time.
  - **Checkpoint errors**
    - Don't delete only part of the checkpoint dir; if you change schema/output mode, **clear both** the sink path and checkpoint to start clean.
- 

## 8) Useful commands

```
# Watch Kafka logs
docker compose logs -f kafka
```

```
# Watch the Spark app logs (console sink output is here)
docker compose logs -f spark-app
```

```
# Restart only the Spark app (after code changes)
docker compose restart spark-app
```

```
# Stop everything
docker compose down
```

```
# Stop and remove volumes (CAUTION: clears checkpoints & data)
docker compose down -v
```